# Artificial Intelligence Software, Lecture 2

## Introduction to VibeCoding


# Concept: Exoskeleton for the Mind

- **Reducing Cognitive Load:** No need to memorize all syntax or CLI parameters.
- **Accelerating Routine:** Automating what you already know but do slowly.
- **Focus on Essence:** Shifting from ”how to write” to ”what to write”.


*”AI will not replace programmers, but programmers with AI will replace those without it.”*

# Practical Use Cases

## Boilerplate Generation

- Infrastructure: Docker Compose, Ansible playbooks.
- Data Mappers.

## Complex Syntax

- Regular Expressions (Regex).
- Complex SQL queries (Window functions, CTEs).
- Bash scripts and awk/sed magic.

# Prototyping

- Testing hypotheses in 5 minutes instead of an hour.
- Rapid MVP creation for idea validation.
- Translating code from one language to another (e.g., Python → C++).

Important: Prototype ≠ Production Code. But the path to it becomes shorter.

# Reference Materials

- [ENG] Github Copilot: Impact on developer productivity

# LLM as a Prediction Engine

An LLM is not a database of facts, but a probabilistic distribution over the next token.

$$P(token_{t+1} | token_1, \ldots, token_t)$$

- **Token:** A piece of word (e.g., ”ing”, ”the”, ” import”). 0.75 words.
- The model assigns a probability to every possible next token.
- **Inference:** We sample from this distribution to generate text.

# Sampling Parameters: Temperature

**Temperature ($T$):** Controls the ”creativity” (randomness) of the model.

- **Low $T$ ($0.1 — 0.3$):** Peaked distribution. Selects the most probable tokens.
  - *Use for:* Coding, Math, formatting JSON.
- **High $T$ ($0.7 — 1.2$):** Flatter distribution. Allows less probable tokens.
  - *Use for:* Brainstorming, Storytelling, Creative writing.

[ENG] Temperature in action (Video)

# Sampling Parameters: Top-k & Top-p

Strategies to avoid ”garbage” tokens from the long tail.

**Top-k:**

- Limits choices to the top $K$ most likely tokens (e.g., $K = 50$).
- Hard cutoff.

Top-k video

**Top-p (Nucleus):**

- Selects smallest set of tokens whose cumulative probability exceeds $P$ (e.g., $P = 0.9$).
- Dynamic cutoff (adapts to confidence).

# Why Hallucinations Occur?

## Stochastic Nature

The model completes a *pattern*, not retrieves a *fact*. If the most statistically probable completion for ”Library for X” is ”X-Lib-Pro” (which doesn’t exist), the model generates it.

## Danger Zone

When the model doesn’t know the answer, it ”dreams” up a plausible-sounding one to satisfy the pattern of a helpful assistant.

# Evolution of Interaction

From simple copy-paste to autonomous agents.

1. **Chat-based:** Manual context, Copy-Paste.
2. **Inline / Autocomplete:** ”Flow” in the editor.
3. **Context-Aware:** Understanding the whole project.
4. **Agentic:** Turnkey task execution.

# Level 1-2: Chat and Autocomplete

## Level 1: Chat (ChatGPT Web)

- Pros: Access to the smartest model.
- Cons: No project context, manual copying.

## Level 2: Inline (Copilot Ghost Text)

- Pros: Does not break the flow (Flow state). FIM (Fill-In-Middle).
- Cons: Works only in local context (current file).

# Level 3-4: Agents

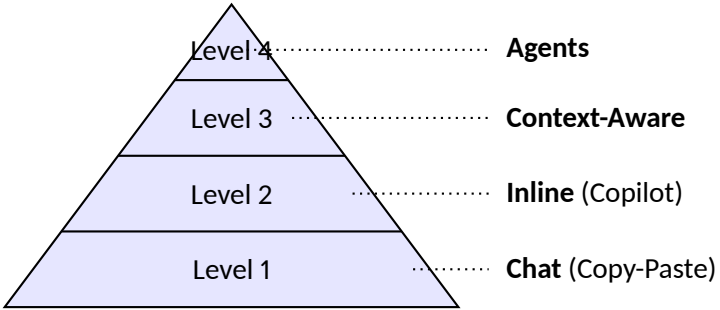
## Level 3: @Workspace Chat

- RAG over the project. ”Where is this class defined?”, ”How are these modules connected?”.

## Level 4: Agentic (Cursor Composer / Windsurf)

- ”Make frontend for our trained model”.
- AI opens files, edits, runs terminal commands itself.

# Hierarchy (Visual)




Pyramid of AI Autonomy in Development

# Task Separation

- **High variability, low risk:** Ideal for AI.
- **Low variability, high risk:** Requires careful control.

Not all tasks are equally beneficial to delegate to AI. Blind trust leads to ”future technical debt”.

# Green Zone (YES) - ML/Robotics

- **Data Preprocessing:** Pandas/Numpy pipelines, data augmentation scripts.
- **Model Boilerplate:** Standard PyTorch/TensorFlow training loops, DataLoaders.
- **Visualization:** Matplotlib/Seaborn plots for EDA and metrics.
- **Infrastructure:** Dockerfiles for CUDA envs, ROS launch files/URDFs.
- **Documentation:** Explaining model architectures and math formulas.

# Red Zone (NO/Caution) - ML/Robotics

- **Novel Architectures:** Designing SOTA models without understanding (hallucinated layers).
- **Safety-Critical Control:** Robot kinematics/dynamics where errors cause physical damage.
- **Subtle Math Errors:** Tensor dimension mismatches, incorrect gradients in custom loss functions.
- **Proprietary Data:** Never paste private datasets or patient data into the prompt.

# Multiplier Effect

AI acts as a multiplier of your current qualification.

$$Result = Skill \times AI\_Factor$$

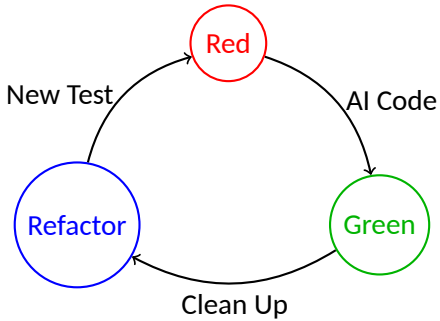
- Senior ($10 \times 1.5 = 15$): Acceleration, focus on architecture.
- Junior ($2 \times 1.5 = 3$): Help in learning, but risk of ”illusion of knowledge”.
- Zero ($0 \times 1.5 = 0$): Without a base, AI is useless for serious tasks.

# Methodology 1: TDD + AI (Synergy)

**Test Driven Development:**

1. You write a test (behavior specification).
2. Test fails (Red).
3. AI writes implementation to pass the test (Green).
4. You refactor code with AI (Refactor).

*Example:* "Write a test for a function that IOUs (Intersection over Union) of two bounding boxes."




TDD Cycle

# Methodology 2: BDD & RDD (Specification-Driven Development)

## Behavior Driven Development (BDD)

**Concept:** Describe scenarios in plain language.

*Example:* "Given a loaded YOLOv8 model, When input is a black image, Then 0 objects should be detected."

→ AI translates this into 'pytest-bdd' code effortlessly.

## Readme Driven Development (RDD)

**Concept:** Write the 'README.md' with usage examples first.

AI uses the Readme as a specification to implement the internal logic (API, CLI arguments).

# Methodology 3: Type-First Design

**Idea:** Define complex types and interfaces before logic.

- Use Pydantic/Dataclasses to define schemas (e.g., ‘ModelConfig‘, ‘InferenceResult‘).
- AI is excellent at ”filling in the blanks” if the types are strict.
- *Robotics:* Define ‘Translation3d‘ and ‘Rotation3d‘ types, let AI write the transformation logic.

# Skills to Boost Vibe Coding

To use AI effectively, shift focus from syntax to:

- **System Design:** Modular architecture (Clean Arch). AI can write the function, but you define where it lives.
- **Code Review:** You read 10x more code than before. Spotting subtle broadcasting errors in PyTorch is a key skill.
- **Property-Based Testing:** Description of properties: ”The output probability distribution must always sum to 1.0”. (Refs: ‘Hypothesis‘).

# Property-Based Testing (Deep Dive)

## Concept

Instead of: *"If we input X, we get Y"*

You formulate: *"For any valid input data, property Z must hold."*

- **Example (Sorting):**
  - *Standard:* sort([3,1,2]) == [1,2,3]
  - *Property:* Output is sorted AND contains same elements as input.
- **Why use it?**
  - **Edge Cases:** Tools (like Hypothesis) automatically find hard-to-spot bugs (NaNs, empty lists).
  - **AI Synergy:** AI implements the logic, you define the physics/math **invariants** (e.g., "Probability sum = 1.0").

# Danger for Juniors

- Skipping ”Basics”: If you don’t write bubble sort by hand at least once, you won’t understand algorithm complexity.
- ”StackOverflow Effect” on Steroids: Copying code without understanding.
- Debugging Complexity: Alien code (even from AI) is harder to debug than your own.

# Reference Materials

- [ENG] Kent Beck: TDD with GitHub Copilot
- [ENG] Hypothesis: Property-based testing for Python
- [ENG] Readme Driven Development
- [ENG] Spec-Driven Development (Video)
- Clean Code and Clean Architecture – Robert C. Martin

# Garbage In, Garbage Out

The quality of the model's answer is directly proportional to the quality of context and prompt.

- Charity of Intent.
- Constraints.
- Output Format.

# Core Techniques

1. **Role Playing:** ”Act as a Senior Python Developer specialized in FastAPI...”
2. **Chain-of-Thought:** ”Think step by step. First, analyze the structure...”
3. **Few-Shot Prompting:** ”Here is an example of my code. Create a new function in the same style.”

# Iterativity

Don’t try to get perfect code in one prompt.

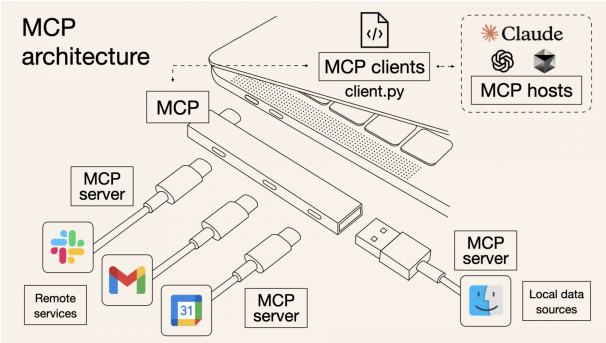
- Step 1: Describe the solution plan.
- Step 2: Implement class structure.
- Step 3: Write methods.
- Step 4: Write tests for this.

# Reference Materials

- [ENG] OpenAI Prompt Engineering Guide
- Как писать промпты от Google (перевод)

# The Isolation Problem

LLMs are usually locked in a chat ”room”. They don’t see your database, can’t access Jira, can’t read server logs in real-time. **MCP** is a USB port for AI models.



# MCP Architecture

- **MCP Host:** The application where you work (IDE: Cursor, Windsurf, Claude Desktop).
- **MCP Client:** Agent inside the host.
- **MCP Server:** Bridge to data (PostgreSQL, Google Drive, Local Filesystem).


MCP Connection Scheme

# Use Cases

- ”Check stock in Postgres DB and write an API endpoint”.
- ”Find requirements for this module in documentation on Google Drive”.
- ”Analyze error.log and propose a fix”.
- ”Open Blender, create a cube, and render it”.

# Top Useful MCP Servers

- **Context7 (Documentation):**
  - Real-time access to docs for libraries (e.g., "How to use generic types in Pydantic v2?").
  - Instead of hallucinating, the model searches valid documentation (RAG over the web).
- **Puppeteer / Playwright (Browser Control):**
  - Control a real browser: "Go to localhost:3000, login as admin, and take a screenshot."
  - Essential for generating and debugging End-to-End (E2E) tests.
- **Filesystem (Local Access):**
  - Allows the agent to read logs (‘/var/log/syslog‘) or configs outside the project folder.

# Reference Materials

- [ENG] What does MCP means
- [ENG] Smithery: MCP for everyone
- Полный гайд по MCP (Видео)

# Difficulties and Barriers

Most services (OpenAI, Anthropic, GitHub Copilot) require foreign cards and IP. Account bans.

- Obtain edu version of Github Copilot.
- [RUS] MTS Payment
- Self-hosting (Ollama) as a workaround.

# Reference Materials

- Ollama приложение
- Ollama cli

# Best models

*Video*

# Experimental Setup: Tools and Environment

- **Development Environments:**
  - **Cursor:** Used for 6 out of 9 models available out-of-the-box .
  - **Cline:** Used for models requiring API integration via **Polza AI** aggregator .
- **Model Selection:**
  - **Anthropic:** Sonnet 4.5, Opus 4.5 .
  - **Google:** Gemini 3 Pro, Gemini 3 Flash .
  - **OpenAI:** GPT 5.2 (with *Extra High Thinking* mode) .
  - **Others:** Composer (Cursor), Kimi K2, GLM 4.7, Qwen 3 Max .

# Methodology: The "Vibe Coding" Rules

- **Initial Prompt:** Each model receives one comprehensive starting prompt describing the entire project.
- **Iterative Debugging:**
  - If the output has issues, the model is allowed up to **5 additional calls** to fix bugs .
  - Results are finalized after the 5th correction attempt.
- **Evaluation Criteria:** Ability to build complex MVPs (Fantasy To-Do lists, Reddit scrapers, Admin dashboards, Video dubbing pipelines) from scratch.

# Key Findings: Best Performing Models

- **GPT 5.2 (Overall Winner):**
  - Won all tests decisively .
  - Success is attributed to the **Extra High Thinking** mode, prioritizing reasoning time over speed .
- **Claude Opus 4.5:**
  - Exceptional logic and high-quality UI design .
  - Performance is nearly on par with the leader, often working ”perfectly” with minimal fixes .

# Key Findings: Budget and Niche Solutions

- **Gemini 3 Flash (Best Value):**
  - Recognized as the best balanced choice for ”budget AI coding”.
  - Offers a strong performance-to-cost ratio.
- **Composer (Fastest):**
  - The quickest model for small, specific tasks .
  - Less effective for building entire systems from zero .

# Final Conclusions

- **Thinking vs. Result:** There is a clear correlation between the model's reasoning time and the quality of the final code .
- **Shift in Development:** The bottleneck in 2026 is no longer writing code, but **architecture planning, documentation study, and algorithm design**.
- **One-Prompt MVPs:** It is now viable to create fully functional startup MVPs with a single well-structured prompt.

# IDE-Based Tools

- **Cursor (VS Code Fork)**
  - **Pros:** Deep codebase understanding (200k context), background agents, multi-file editing.
  - **Cons:** Steeper learning curve, recently changed pricing (removed unlimited plans).
- **Windsurf (Codium)**
  - **Pros:** ”Flow-aware” development, excellent context management, fast response times.
  - **Cons:** Credit-based usage can be confusing, some find completion slower than Cursor.

# Official Ecosystems: Claude Code & Copilot

- **Claude Code (Anthropic CLI)**
  - **Pros:** Most advanced logic (Sonnet 4.5/Opus 4.5), no hidden fees, ”Thinking Mode”.
  - **Cons:** CLI-based (lack of GUI for some), can burn tokens quickly in complex tasks.
- **GitHub Copilot (Microsoft)**
  - **Pros:** Most affordable (10/*mo*), native VS Code integration, very stable.
  - **Cons:** Agent mode can be slower/less powerful than specialized rivals.

# Specialized Agents: Goose & Cline

- **Goose (Block/GitHub)**
  - **Pros:** Open-source client, supports any model via OpenRouter/Local LLMs.
  - **Cons:** Slower process, purely agentic (autonomous) approach might be harder to control.
- **Cline (formerly Kodu)**
  - **Pros:** Full transparency in costs, checkpointing (rollback feature), "Bring-Your-Own-Key" (BYOK).
  - **Cons:** Requires manual API management, strictly agent-focused.

# Comparison Summary

| Tool | Key Strength | Main Weakness |
|---|---|---|
| **Cursor** | Deep Context/Refactoring | Complex UI/Pricing |
| **Claude Code** | Best Reasoning/Thinking | CLI-only interface |
| **Windsurf** | Flow/UX Speed | High Credit Consumption |
| **Copilot** | Affordability/Integration | Limited Agent Power |
| **Bolt/V0** | Rapid Prototyping (Vibe) | Hard to maintain long-term |

# Conclusions: The Shift to Agentic Coding

- **From Assistant to Agent:** Tools now plan, run terminal commands, and debug themselves.
- **The Bottleneck:** Reasoning (Thinking) is now more important than raw speed for complex MVPs.
- **Educational Risk:** Over-reliance on agents might hinder learning fundamentals for beginners.
- **Best Strategy 2026:** Use Claude Code for logic, Cursor/Windsurf for codebase navigation, and Gemini Flash for budget tasks.

# Reference Materials

- Claude Code: полный гайд по AI-кодингу (хаки, техники и секреты)
- We Tier Ranked Every AI Coding Assistant
- Windsurf vs Copilot vs Cursor In 2026 (Pros, Cons, & Pricing Update)
- ИИ Агент Для Программирования — КАК ВЫБРАТЬ? (Cursor, Windsurf, Goose, VS code, Cline...)

# Deserve “A” grade!

– Oleg Bulichev

bulichev.ov@mipt.ru

@Lupasic

Цифра, 521